2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00066

Prosperity: Accelerating Spiking Neural Networks via Product Sparsity

Chiyue Wei†, Cong Guo†*, Feng Cheng†, Shiyu Li†, Hao (Frank) Yang‡, Hai (Helen) Li†, Yiran Chen†
Duke University†, Johns Hopkins University‡
{chiyue.wei, cong.guo, feng.cheng, shiyu.li, hai.li, yiran.chen}@duke.edu, haofrankyang@jhu.edu

***Abstract*—Spiking Neural Networks (SNNs) are highly efficient due to their spike-based activation, which inherently produces bit-sparse computation patterns. Existing hardware implementations of SNNs leverage this sparsity pattern to avoid wasteful zero-value computations, yet this approach fails to fully capitalize on the potential efficiency of SNNs. This study introduces a novel sparsity paradigm called *Product Sparsity*, which leverages combinatorial similarities within matrix multiplication operations to reuse the inner product result and reduce redundant computations. Product Sparsity significantly enhances sparsity in SNNs without compromising the original computation results compared to traditional bit sparsity methods. For instance, in the SpikeBERT SNN model, Product Sparsity achieves a density of only 1.23% and reduces computation by $11\times$, compared to bit sparsity, which has a density of 13.19%. To efficiently implement Product Sparsity, we propose *Prosperity*, an architecture that addresses the challenges of identifying and eliminating redundant computations in real-time. Compared to prior SNN accelerator PTB and the A100 GPU, *Prosperity* achieves an average speedup of $7.4\times$ and $1.8\times$, respectively, along with energy efficiency improvements of $8.0\times$ and $193\times$, respectively. The code for *Prosperity* is available at https://github.com/dubcyfor3/Prosperity.**

***Index Terms*—Spiking Neural Networks, Accelerator, Product Sparsity**

I. Introduction

Spiking Neural Networks (SNNs) [27], [61] are brain-inspired neural network models that have gained significant attention in recent years. Extensive research efforts in SNN algorithm design have yielded performance comparable to Deep Neural Networks (DNNs) across various tasks, including computer vision [6], [19] and natural language processing [2], [60]. Beyond traditional AI applications, SNNs show promise in rapidly solving optimization problems [65] and heuristically approaching NP-hard problems [45], further underscoring their versatility and potential impact across diverse computational domains.

Compared to the traditional DNN models, as shown in Fig. 1 (a), the SNN's neurons only react to information encoded as binary spikes (1 for spike and 0 for non-spike), as depicted Fig. 1 (b), leading to sparse activations (i.e., ***bit sparsity***). This design leads to significant computational and energy efficiency gains [71], which arise from the spike-driven sparse processing in SNNs, depicted in Fig. 1 (c). Consequently, many SNN-specific architecture designs [52], [66] leverage this *bit sparsity* to enhance performance and energy efficiency, establishing a paradigm for bit sparsity-based accelerators. Nevertheless, our study observed that *bit sparsity* alone does not fully harness the potential sparsity within SNNs.

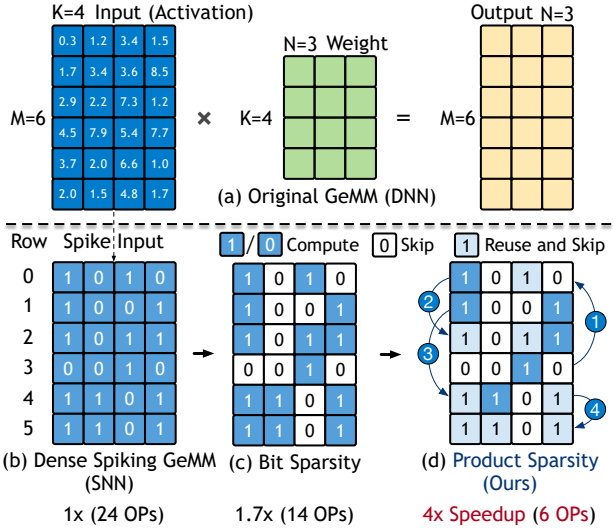

Fig. 1. Comparison of DNN, SNN, Bit Sparsity, and Product Sparsity.

In this study, a novel sparsity paradigm, ***Product Sparsity (ProSparsity)***, is proposed, which can reveal additional sparsity opportunities within SNN models, as illustrated in Fig. 1 (d). ProSparsity is founded on the combinatorial similarity within the inner product operations of matrix multiplication. Specifically, in SNNs, multiple rows of the binary spike matrix usually contain a common binary sub-combination, resulting in identical inner product results when multiplied by the shared weight matrix. By computing the inner product of the sub-combination only once and reusing the result for all rows containing the same sub-combination, ProSparsity effectively eliminates the repeat computations of spikes within the common sub-combination. Consequently, ProSparsity significantly enhances sparsity and reduces redundant computations compared to traditional bit sparsity.

For example, in Fig. 1 (d), Row 1 (1001) and Row 4 (1101) share the common sub-combination 1001. Thus, the inner product result of Row 1 (1001) can be reused for Row 4, with additional computation needed only for the remaining sub-combination 0100, saving two operations. Moreover, since Row 5 is identical to Row 4, all computations for Row 5 can be skipped, and the result of Row 4 can be directly applied to

*Cong Guo is the corresponding author of this paper.

979-8-3315-0647-625/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00066

Row 5. Consequently, ProSparsity enables significant computation skipping and result reuse. Our experiments demonstrate that, in the SpikeBERT SNN model [60], ProSparsity achieves a $\mathbf{11}\times$ reduction in computation. In comparison, while bit sparsity achieves a density of $\mathbf{13.19}\%$, ProSparsity reduces the density to just $\mathbf{1.23}\%$.

To the best of our knowledge, this is the first work to accelerate SNNs with product sparsity, thereby leading to several significant challenges in designing a ProSparsity-based architecture. Identifying common sub-combinations presents both spatial and temporal challenges. Firstly, for spatial relationships, ProSparsity must identify the combinatorial similarity between multiple rows. The complexity for identifying spatial relations among n rows within an m-row scope is $O(m^n)$, which is not feasible at runtime. Therefore, an efficient method is needed to identify ProSparsity relations with manageable overhead. Secondly, temporal relationships also significantly impact the efficiency of ProSparsity. For instance, in Fig. 1 (d), if Row 0 (1010) is processed first, it cannot reuse the result from Row 3 (0010). Improper execution orders will significantly undermine the effectiveness of ProSparsity. Thus, optimizing the execution order of spike rows is crucial for maximizing reusability and achieving high sparsity. Moreover, since spike activations are generated dynamically, the identification of sub-combinations and the elimination of redundant computations have to be done at runtime, which further complicates the design of ProSparsity.

To address these challenges, we propose ***Prosperity***, an architecture designed to efficiently process SNNs using product sparsity. Based on our analysis, we first introduce a heuristic optimization to reduce the time and space complexity of ProSparsity processing to a linear level, addressing both spatial and temporal challenges. We then design an efficient architecture and pipeline scheduling mechanism that overlaps the identification and elimination of ProSparsity with matrix computation, resulting in overhead-free sparse processing. Furthermore, since ProSparsity is algorithm-agnostic, *Prosperity* can support a wide range of SNNs, including emerging spiking networks such as spiking transformers [2], [94], which are not supported by existing accelerators [52], [58], [62].

Compared to state-of-the-art SNN accelerators like PTB [52] and the A100 GPU [67], *Prosperity* achieves an average speedup of $7.4\times$ and $1.8\times$, respectively, along with energy efficiency improvements of $8.0\times$ and $193\times$, respectively. Our contributions are summarized as follows:

- We propose a novel sparsity paradigm, ***Product Sparsity***, that utilizes the combinatorial similarity feature in SNN activations to significantly enhance sparsity and reduce computations in SNNs.
- To tackle high complexity and dependency challenges in ProSparsity, we present a dedicated architecture, ***Prosperity***, that efficiently identifies and utilizes ProSparsity within linear complexity.
- We design an efficient pipeline scheduling method to further overlap the ProSparsity processing with the matrix computation, achieving overhead-free sparse processing.
- Finally, *Prosperity* architecture effectively accelerates a wide range of SNNs, achieving significant speedup and energy efficiency over baselines.

II. Background

A. Spiking Neural Networks

Spiking Neural Networks (SNNs) [27], [72], said to be the third generation of neural networks [61], mimic the behavior of biological brain neural networks. SNNs have a similar structure as traditional Deep Neural Networks (DNNs) [29], [50] but differ in two aspects: 1) the information (activation) propagating in the network is a binary spike event in a time sequence instead of a floating point number. 2) the activation function is replaced by the membrane potential (MP) update [26] and spike fire behavior of the spiking neuron. A single forward pass of SNN is accomplished in a certain number of time steps. In each time step, a spiking neuron receives spikes from neurons in the previous layer and then integrates received spikes according to the corresponding weight to get input current; it then uses the input current to update its MP and send a spike to neurons in the next layer if its MP exceeds a certain threshold. There are various spiking neuron model [25], [26], [39], [43]. We consider the most widely used leaky integrate and fire (LIF) model [26].

SNNs are more energy efficient than DNNs due to their binary activation. In DNN, the major computation is the matrix multiplication between floating point activation and weight matrix, i.e., general matrix multiplication (GeMM). In SNN, the activation of an SNN layer consists of a set of binary spike matrices, each representing a time step of SNN and sharing the same weight. Therefore, we can unroll and concatenate all spike matrices in different time steps to get a single binary spike matrix for an SNN layer. The major computation of the SNN model is the matrix multiplication of the binary spike matrix and floating point weight matrix [16], [71]. We define this unique operation in SNNs as **Spiking GeMM**. Our experiments show that more than 98% percent of operations in SNNs are spiking GeMM. In spiking GeMM, the binary spike matrix only consists of 0s and 1s, i.e., ***bit sparsity (BitSparsity)***. Therefore, the computation becomes a sparse addition. If an element is 1, then the corresponding weight is straightly accumulated to the result; if an element is 0, then the corresponding weight is skipped. Prior works [52], [57], [66], [70] leveraged the sparse and multiplication-free feature in spiking GeMM to accelerate SNN.

B. Spiking CNNs and Spiking Transformers

The most widely used network structure in SNNs is convolutions, which is spiking CNNs [19], [74]. Besides, SNNs in transformer architecture have developed very recently while demonstrating superior performance than spiking CNNs on various tasks [2], [60], [80], [84], [94].

The spiking CNNs and spiking transformers differ from their DNN counterparts in the two aspects mentioned in Sec. II-A. The key operation in spiking CNNs and spiking

TABLE I
COMPARSION WITH PREVIOUS WORK ON VGG-16.

Study	Dense	PTB [52]	Stellar [62]	***Prosperity***
Feature	-	Long Steps	Specific Neuron	**Algorithm-agnostic**
Sparsity Pattern	None	Structured BitSparsity	Structured BitSparsity	**Unstructured ProSparsity**
Bit Density	100%	34.21%	9.80%	34.21%
Pro Density	100%	-	-	**2.79%**
Speedup	1.00×	1.86×	5.97×	**17.55**×

transformers is spiking GeMM. In spiking CNNs, the convolution is performed between a spike input feature map in multiple time steps and a weighted kernel, generating an output feature map in multiple time steps. This operation can be translated to spiking GeMM by im2col [7].

In spiking transformers, the linear projection layers (query, key, value, output projection, and feed-forward layer) are naturally spiking GeMM, where spike matrix of shape $(T \times L, d_i)$ is multiplied with weight matrix of shape (d_i, d_o) and get output matrix of shape $(T \times L, d_o)$, where T, L, d_i, d_o denotes time steps, sequence length, input dimension, and output dimension. Different spiking transformer models incorporate diverse spiking attention blocks that are distinct from linear layers. Consequently, operations in spiking attention are not efficiently supported by existing SNN ASICs [52], [62], [66] or neuromorphic chips [14], [15].

C. SNN Accelerators

Previous SNN accelerators have been exploring utilizing the bit sparsity of SNNs for efficient inference, as summarized in Tbl. I. PTB [52] is a systolic-array-based [49] architecture that parallelly processes spikes in a structured sparsity manner. It employs time batching [66], and further group spike information within a time window. If a spike occurs in a group, all time steps within that window are processed, while groups without spikes are squeezed out to enhance utilization of systolic array for sparse input.

Stellar [62] is an algorithm-hardware co-design accelerator. It is also a systolic-array-based architecture that process spikes in a structured sparsity manner. Stellar proposes using FS neuron [77] model in the SNNs; this FS neuron is featured to have fewer spikes than the LIF neuron, achieving higher spike sparsity. The improvement of sparsity is caused by algorithmic modification, which limits the application scenarios of the accelerator.

III. PRODUCT SPARSITY

In this section, we first introduce the product sparsity in spiking GeMM. Then, we describe some basic definitions and propose an efficient optimization for product sparsity, significantly reducing its complexity.

A. Definition

In the spiking GeMM, the binary nature of the spike matrix inherently introduces bit sparsity, i.e., the zero-value operand

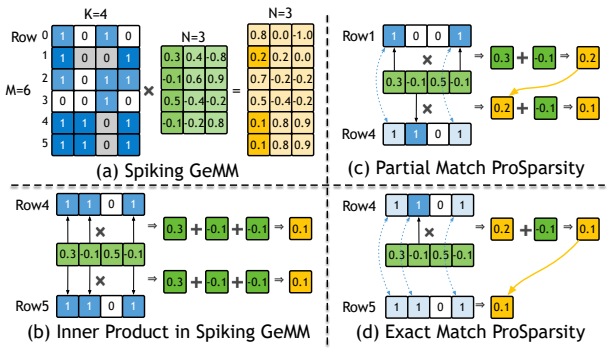

Fig. 2. An example of spiking GeMM and product sparsity

can be skipped during computation. However, there are more redundant computations beyond the zero-value operand. To demonstrate this redundancy, we illustrate a spiking GeMM in Fig. 2 (a). For a $M \times K$ spike matrix, a $K \times N$ weight matrix, and a $M \times N$ output matrix, the spiking GeMM can be computed by multiple BitSparsity-based inner product operations, shown in Fig. 2 (b). Due to the binary value of the spike matrix, the inner product is equivalent to the accumulation of multiple weight values, which are "selected" by the 1-values in the corresponding row of the spike matrix. Based on the arithmetic rule of GeMM, the i-th column of the spike matrix will "select" the identical i-th row of the weight matrix in the inner product operations. Two rows in the spike matrix contain a common binary sub-combination if they share identical 1-value positions, either partially or fully. Consequently, the common binary sub-combination will generate the same inner product result when computing the same column of the output matrix. For example, in Fig. 2 (b), Row 4 and Row 5 have the same result because they have the identical binary combination (1101). Obviously, we can compute the common binary sub-combination once and reuse its result for all rows containing this sub-combination, thereby significantly reducing computational redundancy in spiking GeMM. We call this paradigm ***product sparsity (ProSparsity)***, which can skip computation and reuse the result based on the combinatorial similarities of inner product operation.

As shown in Tbl. I, ProSparsity completely differs from the previous approaches, which depend on the algorithm and may impact the accuracy. ProSparsity is an algorithm-agnostic and lossless method for the spiking GeMM. Based on the potential combinatorial similarities in the spike matrix, ProSparsity can significantly reduce computations. For the VGG-16 SNN model, ProSparsity can save more than $18\times$ computations and achieve $9.4\times$ speedup over the BitSparsity of PTB [52].

Despite ProSparsity providing a simple yet effective way to identify the redundancy in SNN, its implementation is a nontrivial task, as mentioned in Sec. I. There are two main challenges in processing ProSparsity: Efficiently identifying spatial and temporal relationships. We will introduce each challenge and provide an efficient heuristic solution to address these challenges.

B. Spatial Relationship

For ProSparsity, it will cause significant complexity to identify the combinatorial similarities. Assuming we have m rows and aim to identify the common sub-combination for n rows, the resulting complexity will be $O(m^n)$. For example, if we want to find the common sub-combination between only two rows within m row, we need to match $\frac{m\times(m-1)}{2}$ times to finish all identification for every two rows. For a better illustration, we use a set to represent the spike row. We define S_i as the spike set of the spike row i; the spike set consists of the column index where a spike exists

$$S_i = \{j | M[i,j] = 1, j \in \text{Column IDs}\},$$

where M is the spike matrix. For example, Row 0 (1010) in Fig. 2 (a) can be represented by $S_0 = \{0, 2\}$. The identification process is to inspect the set relationship among the n sets represented by n rows, called spatial relationship. Basically, when $n \geq 3$, the complexity of the identification will be unacceptable. Therefore, in this study, we only consider the two-row spatial relationship, which can achieve high enough sparsity for SNNs, with the complexity $O(m^2)$.

Basically, we discover 3 types of ProSparsity relationship when two sets are intersected, $A = S_i \cap S_j, A \neq \phi, i \neq j$. We will define 3 relationships with the intersection set A.

Partial Match (PM). We define the Partial Match (PM) relationship for two spike rows as follows:

$$A = S_j, A \neq S_i.$$

This indicates that S_j is the proper subset of S_i. For example, in Fig. 2 (c), Row 1 (1001) is the proper subset of Row 4 (1101). Consequently, the spike row S_4 (Row 4) can reuse the inner product result (0.2) of S_1 (Row 1). After that, for S_4, we can skip the spikes in S_1 and accumulate the rest weight values (-0.1) that correspond to spikes in $S_4 - S_1$ (0100).

Exact Match (EM). Exact Match product sparsity means two rows are identical, i.e.,

$$A = S_i = S_j.$$

As exemplified in Fig. 2 (d), we have computed the inner product result (0.1) of Row 4 (1101); we can reuse it as the result of Row 5 (1101) and skip all the spikes without any computation.

Intersection. Finally, when $A \neq S_i$ and $A \neq S_j$, that means the two rows have an intersection relationship. However, leveraging this scenario requires creating a new row A and compute the result of A first. This will significantly increase the complexity of the architecture design. Hence, this study will only consider the former two relationships, EM and PM.

C. Temporal Relationship

The temporal relationships can significantly impact efficiency. For instance, in Fig. 2 (a), if Row 0 (1010) is processed first, it cannot reuse the result from Row 3 (0010), which significantly undermines the effectiveness of product sparsity. Therefore, optimizing the execution order of spike rows is crucial for maximizing sparsity.

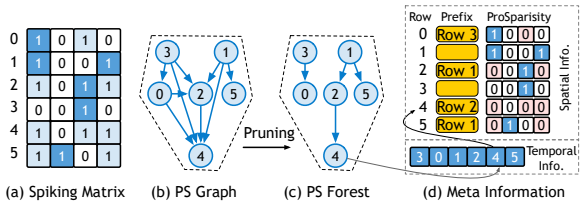

Fig. 3. Efficient ProSparsity data structure.

We first introduce the ***Prefix*** and ***Suffix*** for the temporal relationship of ProSparsity. For the two rows with EM, we can define the row with a smaller index as the Prefix, i.e., Row i is the Prefix of Row j, when $i < j$, because the smaller index is always accessed first. In contrast, PM relationships have a strict partial ordering, which leads to strict temporal ordering. The Prefix in the PM relationship is the spike row with fewer elements (ones), which is the proper subset for the other row, i.e., S_j is the Prefix of S_i, when $A = S_j, A \neq S_i$. Correspondingly, the S_i is the Suffix.

After defining spatial and temporal relationships, we can build a ProSparsity graph for the spiking matrix in the next subsection.

D. Efficient Design Space Exploration

Based on the spatial and temporal relationship, the ProSparsity can be constructed as a directed graph, as shown in Fig. 3. However, the graph structure is still too complicated for the ProSparsity architecture design. Thus, we propose a heuristic method, reducing the time and space complexity from $O(m^2)$ to $O(m)$.

ProSparsity Graph. As shown in Fig. 3 (a) and (b), each spike row in the graph is represented by a node, i.e., a spike row. Each directed edge represents the Prefix-based partial ordering relationship, including the EM and PM edges. The direction of the edge represents a prefix node (spike row) points to a suffix node. Graph-based ProSparsity has both the space and time complexity with $O(m^2)$. However, because a node may have multiple prefix nodes, we still need an extra operation to identify whether the various prefixes are available to reuse together, i.e., whether the multiple prefix nodes are disjoint. That will significantly complicate the ProSparsity design. Fortunately, our preliminary results show we can reduce the graph to a tree with a marginal sparsity drop.

Heuristic Space Pruning. We conduct a characteristic study to explore more efficient design space for ProSparsity. There will be a large overhead if we try to utilize two or more prefix nodes. According to preliminary evaluation, our architecture will suffer a 30% performance drop and larger area overhead to accommodate the second Prefix. Moreover, Tbl. II shows the SNN density results after ProSparsity with only one prefix and two prefix nodes. Clearly, the first prefix node contributes to the most computation reduction compared to the second prefix nodes. Moreover, most nodes have only one Prefix, and due to the disjoint limitation of the prefix nodes, less than 6% of nodes can employ the second Prefix. This illustrates that using one Prefix for each node is capable of

TABLE II
PRELIMINARY EXPERIMENTS ON ONE-PREFIX AND TWO-PREFIX.

		SpikingBERT SST-2	VGG-16 CIFAR100
Bit Sparsity	Density	20.49%	34.21%
One-Prefix	Pro Density	2.98%	2.79%
	Prefix Ratio	$56\% \times 1$	$26\% \times 1$
Two-Prefix	Pro Density	2.30%	1.97%
	Prefix Ratio	$53\% \times 1 + 3\% \times 2$	$20\% \times 1 + 6\% \times 2$

eliminating most of the redundancy. Therefore, in our method, we opt to select only one Prefix for each row.

Pruning Rules. Because of the spatial and temporal relationship definition, the ProSparsity graph is a strict partial ordering set. Based on previous analysis, we can make simple pruning rules for the graph to ensure that each node has only one Prefix.

- First, for each current node, we will traverse all of its Prefix nodes. We retain only the edge(s) connected to the Prefix node(s) that have the largest common subcombination (i.e., the largest subset) with the current node, and remove all other edges. That can ensure that the kept Prefix node(s) is the most similar to this current node.
- Then, if the Prefix nodes with the largest subset are multiple, ProSparsity can select any of them. In practice, we keep the edges from the node with largest index.

After this pruning, the ProSparsity graph will become a directed forest called ProSparsity Forest.

ProSparsity Forest. As shown in Fig. 3 (c), the ProSparsity Forest will be the fundamental data structure of our design. After pruning, the ProSparsity Forest may contain multiple directed trees that are independent of each other. Each tree maintains complete partial ordering information for a processing order constraint of spiking GeMM. In spiking GeMM, if we process a spike matrix from the top to the bottom, we may not be able to reuse the result of the prefix row since the prefix result is not computed. The topology order of the product sparsity tree from root to leaves guarantees that each node comes after its prefix, promising high effectiveness for ProSparsity.

Meta Information. In practice, ProSparsity Forest is formed by meta information, which is the real data structure in the processing of ProSparsity, Fig. 3 (d). Meta information has two types: spatial information and temporal information, which correspond to the spatial and temporal relationships. First, the temporal information is a vector with a size of m, which records the execution order of each spike row. The ProSparsity will follow the temporal vector to issue the index of the spike row. After that, we can find the spatial information according to the issued index. The spatial information is a list ordered by the row index, and each element in the list contains the Prefix index and ProSparsity pattern. We can use the spatial information to complete ProSparsity-based computation. The meta information can ensure the ProSparsity can be processed correctly and efficiently.

Finally, through the aforementioned development of the ProSparsity data structure and optimization for design space, the time and space complexity of ProSparsity is reduced to an $O(m)$ linear level.

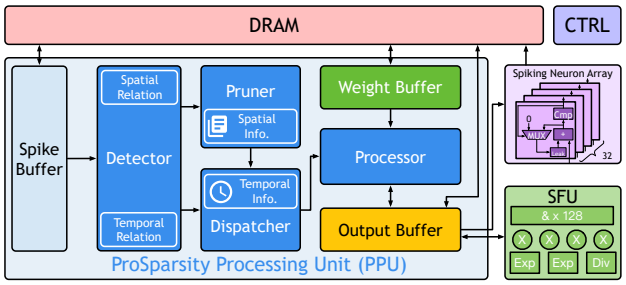

Fig. 4. Overview of *Prosperity* architecture.

IV. *Prosperity* OVERVIEW

Overview. Fig. 4 shows the overall design of *Prosperity*. It mainly consists of the ProSparsity Processing Unit (PPU), Spiking Neuron Array, and special function unit (SFU). *Prosperity* processes the inference of SNNs layer-by-layer. The PPU is our core design for processing ProSparsity and matrix computation for each layer. After that, the results are sent to the Spiking Neuron Array to generate spikes for the next layer, which is a general and necessary unit for SNNs. In this study, we mainly focus on our proposed PPU design. Based on the analysis of the prior section, we can divide the whole processing into four stages: relationship detection, spatial information pruning, meta-information generation, and ProSparsity-based computation, which correspond to the **Detector**, **Pruner**, **Dispatcher**, and **Processor**, respectively.

Detector. The input of the Detector is the spike matrix stored in the spike buffer. The Detector can identify the preliminary spatial and temporal relationships for each combination of two spike rows, which can be used to build the ProSparsity Graph. After that, the Detector will send the original spatial and temporal relationships to the Pruner and Dispatcher to build the ProSparsity Forest.

Pruner. The Pruner will prune the complex spatial relationship to generate a Prefix for each spike row. Pruner is also responsible for generating the ProSparsity pattern. After that, Pruner forms the two pieces of information into meta-spatial information, as shown in Fig. 3 (d), and sends them to the Dispatcher.

Dispatcher. The Dispatcher gathers the spatial information of every spike row and extracts the temporal information according to the temporal relationship. The Dispatcher can construct the ProSparsity Forest with meta information. The ProSparsity Dispatcher leverages the meta information to dispatch the instructions and data to the Processor for the computation of ProSparsity-based spiking GeMM.

Processor. The Processor accesses the weight and output buffer to fetch the corresponding data based on the Prefix and ProSparsity pattern. Based on the temporal information, the Processor can finish inner product computation in the topology order of ProSparsity Forest and store the result in the output buffer. When the latter spike row needs the Prefix,

the Processor can immediately access it from the output buffer. After finishing all computations of a spiking matrix, PPU will send the result to the SFU or Spiking Neuron Array.

Support for Transformers. To support future SNN models, we extend our PPU with an SFU. We reuse the PPU for spiking-GeMM-like operations in spiking attention [2], [94]. With the assistance of the SFU for exponentiation and multiplication in softmax or layer normalization, we support various spiking transformers [2], [60], [84], [94].

V. ProSparsity Processing Unit

In this section, we design different parts of the ProSparsity Processing Unit based on the insight of Sec. III.

A. Spiking GeMM Tiling

Prosperity conducts the tiling method for processing each spiking GeMM in a layer. Tiling of matrix multiplication is a commonly used technique in DNN accelerators [10], [12], [23] that decomposes a large GeMM into several smaller tiles. This approach allows the on-chip buffer to efficiently capture the data reuse and reduce the expensive global memory traffic during the computation of large GeMMs. As shown in Fig. 5, we adopt a tiling size of $m \times n \times k$ in *Prosperity*, where the spike sub-matrix has m rows and k columns, the weight sub-matrix has k rows and n columns, and the output matrix has m rows and n columns. Therefore, we set the corresponding buffer size for on-chip buffers and consider a double buffer mechanism to overlap memory access from DRAM with computation.

However, the tiling strategy for ProSparsity is much more important than that in previous DNN/SNN accelerators because the selection of tile sizes can significantly impact its final sparsity and performance. For example, if we only set $m = 1$, apparently, ProSparsity will be invalid because there is no opportunity to reuse a Prefix. Similarly, the selection of k can also influence the sparsity. However, we cannot select a very large m or k, which will cause excessive area and energy overhead and performance drops. Fortunately, the number of n has no impact on ProSparsity.

Therefore, we build an end-to-end evaluation model to quantitatively discuss the influence of the tile size in detail in Sec. VII-B, considering not only the sparsity but also the hardware performance (speedup) based on our complete *Prosperity* design. Here, we first present the PPU design in the following sub-sections.

B. ProSparsity Detector

Spatial Detection. As discussed in Sec. III-D, the detection of the ProSparsity is to find the PM and EM relationship for spike rows. Therefore, the ProSparsity graph requires $O(m^2)$ complexity to finish detection, which introduces too much runtime overhead. To tackle this challenge, we employ the parallel search ability of Content Addressable Memory (CAM) [68], which is a specialized type of memory that searches its entire stored contents in a single clock cycle to find a match to input query data. CAM is commonly used in network routers [63], data mining [64], and various data-intensive applications [22]. Specifically, we use Ternary CAM (TCAM) [59] to reduce the time complexity to $O(1)$ for each row and $O(m)$ in total for a spike matrix, as shown in Fig. 5 (a). TCAM can match bits with three states: 0, 1, and "don't care" (X), providing flexibility for fuzzy matching. Therefore, we can use the TCAM unit to find all subset indices of a spike row within a single clock cycle.

Step 0, when a spike sub-matrix (tile) is sent to PPU, Detector will pre-load a $m \times k$ spike tile into the TCAM with m entries, each entry with k bits. After initialization, Step 2 and 3 will read each spike row (e.g., Row 2) and mask the one value of the row as 'X' (mask$(1011) = X0XX$). After that, Step 4, TCAM will return all matched entries' indices (i.e., Subset Index, SI). Finally, we realize efficient and parallel spatial information detection with only one cycle.

Temporal Detection. We use the number of spikes in each row as preliminary temporal information, as shown in Fig. 5 (a) 1. To efficiently obtain the number of spikes, we introduce the Popcount [41] operation, which counts the number of ones (NO) in a binary sequence. We place multiple lightweight popcount units in the Detector to obtain this preliminary information.

C. ProSparsity Pruner

The Pruner prunes the spatial relationships to a single Prefix for each row according to our pruning rules.

Efficient Pruning. The Detector provides spatial information (Subset Index, SI) with only subset information without considering the partial ordering underlying the temporal relationship (Number of One, NO). As we discussed in Sec. III-C, for two rows with Exact Match (EM), only the row with the smaller index should be as the Prefix. Therefore, as shown in Fig. 5 (b) 5, we design a proper subset filter for the spike rows with a larger index than the existing query row to eliminate violations according to the partial ordering. Combining SI and NO vectors, we can directly eliminate the EM row with a larger index. For instance, we remove Row 4 from the Prefix candidates for Row 2, where Row 4 has a larger index and is EM to Row 2. After that, we employ an Argmax unit to select only one Prefix (Row 1) for Row 2, based on the heuristic pruning rules in Sec. III-D.

ProSparsity Sparsifying. Consequently, we can utilize the selected Prefix row to generate the ProSparsity pattern, which contains the spike values to be calculated. We use only one XOR (exclusive or) unit to get the ProSparsity for the query row by conducting a bit-wise XOR operation in Step 6. For example, the Row 2 with the Prefix Row 1 can get the ProSparsity pattern: $1011 \oplus 1001 = 0010$. Because the Prefix row is a subset of the query row, the bit-wise XOR is equivalent to the operation $S_q - S_p$.

The spatial information for a query row is now completed and will be sent to the Dispatcher for subsequent usage.

D. ProSparsity Dispatcher

The ProSparsity Dispatcher serves as a link between each part of the unit in PPU, as shown in Fig. 5 (c). It gathers

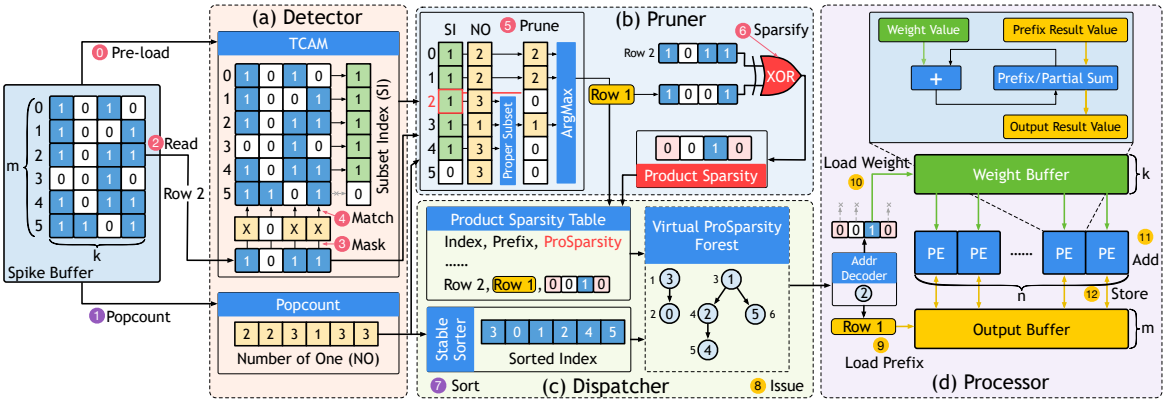

Fig. 5. ProSparsity Processing Unit (PPU) is processing the Row 2 in the ProSparsity spiking GeMM with the tiling size of $m \times n \times k$.

spatial information from the Pruner and stores it in the product sparsity table. It also extracts temporal information according to temporal relationships from the Detector. The Dispatcher extracts the execution order and then dispatches tasks to the Processor. However, there are still some difficulties in generating execution orders under the space and time trade-offs.

Memory Space Issue. Generally, the spatial information contains the complete partial ordering for the ProSparsity Forest. If we record the Prefix and Suffix for each row, we can traverse the Forest to generate an execution order using Breadth or Depth First Search. However, because one spike row may have multiple Suffixes, we should use a matrix with the shape of $m \times m$ to record the two-row spatial information of m rows. Furthermore, the Prefix-Suffix matrix has a large density, as shown in Tbl. II. Even if we adopt a sparse format to store the matrix, the space complexity is still $O(m^2)$ and will lead to irregular accesses. Therefore, the $O(m^2)$ memory space will significantly enlarge the on-chip area more than 10 times compared to the existing design, which is unacceptable for our design. Fortunately, there is only one Prefix for each row. We exploit this feature of ProSparsity Forest to store the spatial information in the product sparsity table with $O(m)$, as shown in Fig. 5 (c). Therefore, we eliminate the suffix information in the tree to achieve space efficiency.

Search Time Issue. However, that will cause the search time issue when we lack Suffix information, i.e., we cannot run Breadth or Depth First Search in a $O(m)$ time. We need to traverse the product sparsity table and then traverse the Forest from the leaf to the root node with $O(m \times d)$ time, where d is the depth of the Forest. In Sec. VII-D, our ablation study shows this way will slow down *Prosperity* by 32%.

Efficient Generation. We propose an interesting and efficient solution to obtain the execution order faster without searching in the product sparsity table. This solution is based on two rules of our design:

- For the Partial Match, the Prefix is the proper subset of the Suffix, i.e., the number of one (NO) in the Prefix is smaller than the Suffix.
- For the Exact Match, the Prefix and Suffix have the same NO. The Prefix has a smaller index than the Suffix.

Therefore, we can use a stable sorting approach to generate the temporal information, as shown in Fig. 5 (c) ⑦. After sorting, we can ensure that the Prefix comes before the Suffix with the temporal information, which is easy to prove. We implement the parallel stable sorter, using a bitonic sorter [4] with $O(log_2^2 m)$ time complexity and $O(m)$ space complexity, which can concurrently conduct the sorting with the Detector and Pruner. In this way, the spatial and temporal information can be completed in the Dispatcher and form a virtual ProSparsity Tree, as shown in Fig. 5 (c).

Then, the Dispatcher then issues tasks (⑧) to the Processor according to the meta information.

E. Product Sparsity Processor

The Processor conducts ProSparsity-based matrix computation according to the guidance of the Dispatcher.

ProSparsity Row-wise Dataflow. *Prosperity* employs a row-wise dataflow, i.e., a spike row with the size of k is processed at a time, and a corresponding output row with the size of n is generated, involving $n \times k$ weight. The Processor processes spike rows in the order provided by the temporal information in the Dispatcher. This order guarantees that the Prefix row in the output matrix is already generated when processing the Suffix row. For each spike row, the Prefix row in the output matrix is fetched and serves as a starting point of the partial sum. Then, we examine the position of the spikes. For each spike, we fetch the corresponding row in the weight matrix and accumulate it on the partial sum. The summed result is written to the corresponding row in the output matrix.

Efficient dataflow implementation. Our Processor design realizes the aforementioned dataflow, as shown in Fig. 5 (d). It mainly consists of a PE array for vectorized accumulation and an address decoder to get data addresses for weight and output buffer. In the process of each row, the Prefix row result is loaded to the partial sum register in the PE array, as shown in Step ⑨ in Fig. 5. Step ⑩ is performed and controlled by the address decoder. This weight address is decoded by bit scan forward [42] operation, which finds the index of the first 1 in the ProSparsity pattern. The decoder then flips this found bit to 0. Step ⑪ is performed by the PE array for accumulation. Step

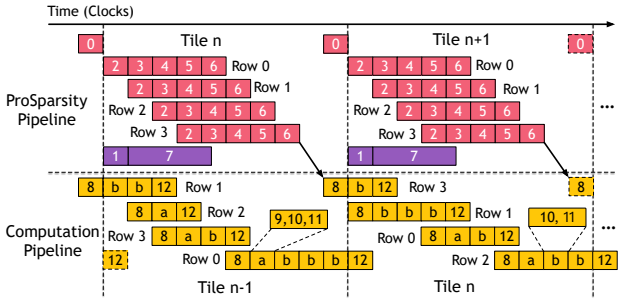

Fig. 6. ProSparsity pipeline scheduling.

10 and Step 11 are repeated until all the 1s in the spike row are consumed, and the result is written back (12) to the output buffer. Since the final result of a tile in the output matrix is the summation of multiple results, this result will also accumulate on the partial sum of the output tile. Our design efficiently bypasses all the zeros in the ProSparsity pattern, supporting unstructured sparsity.

VI. Pipeline Scheduling

We design a pipeline scheduling mechanism to maximize the utilization of our hardware in PPU. A mini example of a pipeline with 4 rows is shown in Fig. 6. The whole PPU can be divided into two phases: the ProSparsity processing phase and the computation phase. The Detector, Pruner, and Dispatcher are responsible for ProSparsity processing, while the Processor conducts the computation phase. These two phases are conducted in a tile-by-tile and sequential manner.

A. Intra-phase Pipeline

The Step 2 to 6 in Detector, Pruner, and Dispatcher is fully pipelined, as illustrated in Fig. 6. Therefore, this pipeline has five stages, achieving the throughput of 1 instruction (spike row) per cycle. Therefore, when the tile has m rows, we only need $m + 4$ cycles for all the spatial information. The temporal information could be extracted (Step 1 and 7) concurrently with Step 2 to 6 pipeline due to our observation that decouples the spatial and temporal information. The Step 7 takes the parallel bitonic sorter with $O(log_2^2 m)$ time complexity to get the sorted temporal information. Thus, the cycles of sorting are far less than m. The pre-load Step 0 can perform concurrently through a double-buffer-based TCAM design. Therefore, the overall cycle for a ProSparsity processing phase is $m + 4$.

For the computation phase, our target is to maximize the utilization of the PE array. The Processor also has four stages in the pipeline: Step 8: issue, Step 9 and 10 for decode and load, Step 11 is execute, and Step 12: write back. For simplicity, we present them into 'a' and 'b', as depicted in Fig. 6. For each spike row, the computation phase may have different cycles due to the spike row having different sparsity. Therefore, for m rows, the overall cycle for a computation processing phase is $\geq m + 4$.

B. Inter-phase Pipeline.

We design an inter-phase pipeline to overlap these two phases. The ProSparsity processing phase is in Tile n, while the Processor runs on Tile $n - 1$ with the meta information already gathered in the Dispatcher. Therefore, the product sparsity table for spatial information adopts a double-buffer design to enable the two-phase overlapping. Based on intra-phase pipeline designs, the cycle of the ProSparsity processing phase is always less than the computing phase. Thus, the ProSparsity processing phase of a tile is perfectly overlapped by the computation phase of the previous tile. Since we conduct tiling and split a large spiking GeMM into multiple tiles, the computation phase can cover almost all of the ProSparsity processing phases except for the first tile phase, which has a minor impact.

Finally, *Prosperity* can realize an overhead-free ProSparsity processing and fully utilize the computation units with the pipeline scheduling.

VII. Evaluation

A. Methodology

SNN models and dataset. We evaluate *Prosperity* on spiking CNNs that previous SNN ASICs design targets. We also evaluate *Prosperity* on different spiking transformers that prior SNN accelerators do not efficiently support. For model architecture, we select VGG [75] and ResNet [38] for spiking CNNs and SpikeBERT [60], SpikingBERT [2], Spikformer [94], SDT [84] for spiking transformers. We use the default configuration for a number of layers, dimensions, and time steps in the paper. These SNN models run on NLP or CV datasets, including SST-2, SST-5, MR, QQP, MNLI, CIFAR10, CIFAR100, CIFAR10-DVS [48], [53], [69], [76], [78], [82]. To get the spike activation pattern in each layer, we run these models implemented in PyTorch. We extract the runtime information and use it in our experiment. *Prosperity* yield iso-accuracy compared to the model in PyTorch as our proposed ProSparsity is an algorithm-agnostic and lossless method.

Hardware Configuration. *Prosperity* configuration is shown in Tbl. III. We implement our core architecture in SystemVerilog [40], including PPU, spiking neuron array, and SFU. The bitwidth of weights is set to 8 bits, which aligns with previous ASICs and neuromorphic chips. We use Synopsys Design Compiler to synthesize our RTL logic to get area and power with ARM's standard cell library under a commercial 28nm process. The spike, weight, and output buffer are evaluated by CACTI 7.0 [3] under a 28nm process to get area and power. The off-chip DRAM power is simulated by DRAMsim3 [54].

Architecture Modeling and Baselines. We build a cycle-accurate simulator to emulate the behavior of *Prosperity* and evaluate the runtime performance. *Prosperity* is benchmarked against dense DNN accelerator Eyeriss [12], SNN accelerators PTB [52], SATO [58], and MINT [87], [88], algorithm-modified SNN accelerator Stellar [62], and NVIDIA A100 GPU. We compare their configuration and performance on VGG-16 with CIFAR100 in Tbl. IV. We maintain the same frequency in every accelerator, while Eyeriss and Stellar have 31% more PEs than SATO, PTB, MINT, and *Prosperity* in

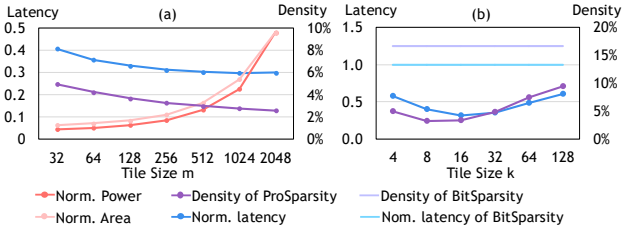

Fig. 7. Performance change with tile size m (left) and k (right).

their design. We simulate Eyeriss, SATO, PTB, and MINT under our simulator framework. We also simulate the For the performance of Stellar [62], we use the statistics reported in their paper since its algorithm modification is not open-sourced, and we are not able to access its sparsity pattern. Since previous SNN accelerators only support the linear layer (e.g., QKV projection, output projection, feed-forward layer) in spiking transformers but cannot deal with attention and layer normalization, we run PTB, SATO, and MINT on the linear layer of spiking transformers for a comparison. We further compare with NVIDIA A100 GPU [67] with 80 GB global memory, running SNN models using PyTorch and SpikingJelly [18], measuring the end-to-end inference time and power by averaging over the whole dataset. Additionally, we include the state-of-the-art SNN accelerator LoAS [86] in our analysis, which focuses on weight pruning and induces dual-side sparse computation. We implement LoAS's algorithm to provide a comprehensive sparsity analysis comparing LoAS and *Prosperity*.

B. Tiling Exploration

We observe that the appearance of product sparsity is related to the tile size of the spike matrix. Therefore, we conduct design space exploration on tile sizes to maximize product sparsity while maintaining a reasonable on-chip buffer size. Fig. 7 shows the relationship between latency/density and the spike tile size in *Prosperity*. The latency is normalized to the latency of the bit sparsity baseline. The statistics are averaged across all spiking CNNs and spiking transformers. We observe that larger m always leads to higher product sparsity, while a suitable k leads to the highest product sparsity. In considering the optimal tile size m, we must balance latency improvements against hardware costs. While a larger m generally reduces latency, it also increases hardware overhead for on-chip buffers and TCAM. The pink/coral lines in Fig. 7 illustrates the relationship between area/power and tile size in *Prosperity*. As m increases, hardware overhead grows super-linearly. Therefore, we should carefully weigh the diminishing returns on latency against the escalating hardware costs when selecting the tile size.

On the M dimension, a larger m means more rows within a tile, providing a larger scope for searching rows that can be used to exploit product sparsity. For each row in the spike matrix, they can look up more rows for the prefix, maximizing the product sparsity.

On the K dimension, a larger k means more columns within a tile, making the spike set of each row bigger and more complicated and, therefore, harder to find a Prefix. When each spike set is small and simple, rows that satisfy equal or subset relationships are more likely to be found. However, as the tile size k gets extremely small, e.g., 4, more rows will appear that have less than two spikes. Exploiting product sparsity for rows that have zero or one spike is meaningless as it does not need accumulation.

According to this exploration, we select a proper tile size for the spike matrix: $m = 256$ and $k = 16$.

TABLE III
Prosperity ARCHITECTURE SETUP.

Tile Size	$m = 256$; $n = 128$; $k = 16$.
Detector	1KB TCAM; 8 Popcounts.
Pruner	256 Channel Subset Unit & Argmax.
Dispatcher	1.5KB Product Sparsity Table.
Processor	128 PEs 8-bit Add, $n = 128$.
Spiking Neuron Array	32 Cells LIF Neuron.
Special Function Unit	128AND/OR; 32MUL; 8EXP; 1DIV.
On-chip Buffer size	8KB Spike; 32KB Wgt; 96KB Out.
DDR4 DRAM	4Gb × 16 2133R 4 Channels 64GB/s.

TABLE IV
COMPARE *Prosperity* WITH PRIOR ACCELERATORS ON VGG-16.

	Eyeriss [12]	SATO [58]	PTB [52]	MINT [87]	Stellar [62]	*Prosperity*
Frequency	500MHz	500MHz	500MHz	500MHz	500 MHz	500MHz
Technology	28nm	28nm	28nm	28nm	28nm	28nm
PEs	168	128	128	128	168	128
ALU	8-b MAC	8-b Add	8-b Add	2-b Add	12b Add	8-b Add
Area (mm^2)	1.068	1.13	-	-	0.768	**0.529**
Throughputs (GOP/s)	29.40 (1.00×)	33.63 (1.14×)	41.37 (1.41×)	62.07 (2.11×)	190.44 (6.48×)	**390.10** (**13.27**×)
Energy Effi. (GOP/J)	16.67 (1.00×)	75.61 (4.53×)	49.70 (2.98×)	34.15 (2.05×)	142.98 (8.57×)	**299.80** (**17.98**×)
Area Effi. (GOP/s/mm^2)	27.53 (1.00×)	29.76 (1.08×)	-	-	247.97 (9.01×)	**737.17** (**26.78**×)

C. End-to-End Performance Analysis

Fig. 8 shows the speedup and energy efficiency of *Prosperity* compared with baselines running different models and datasets. We can also observe superior throughputs and area efficiency of *Prosperity* from the statistics in Tbl. IV. We will compare with other baselines and elaborate on the reasons for our superiority.

Comparison with ASICs. Eyeriss serves as a baseline accelerator that processes SNNs in a dense manner. *Prosperity* achieves a $14.2\times$ speedup and a $21.4\times$ energy efficiency compared with Eyeriss.

Prosperity achieves $7.4\times$ and $4.8\times$ speedup, $8.0\times$ and $4.2\times$ energy efficiency over PTB and SATO for the following two main reasons. These two accelerators trade the sparsity for parallelism when processing spiking GeMM. PTB forces the spike matrix pattern into a structured sparsity during spiking GeMM computation. It processes every bit within a time window as long as one spike exists in the window. This indicates that some zeros are not skipped in PTB. SATO distributes spike rows to multiple PE groups, thus suffering

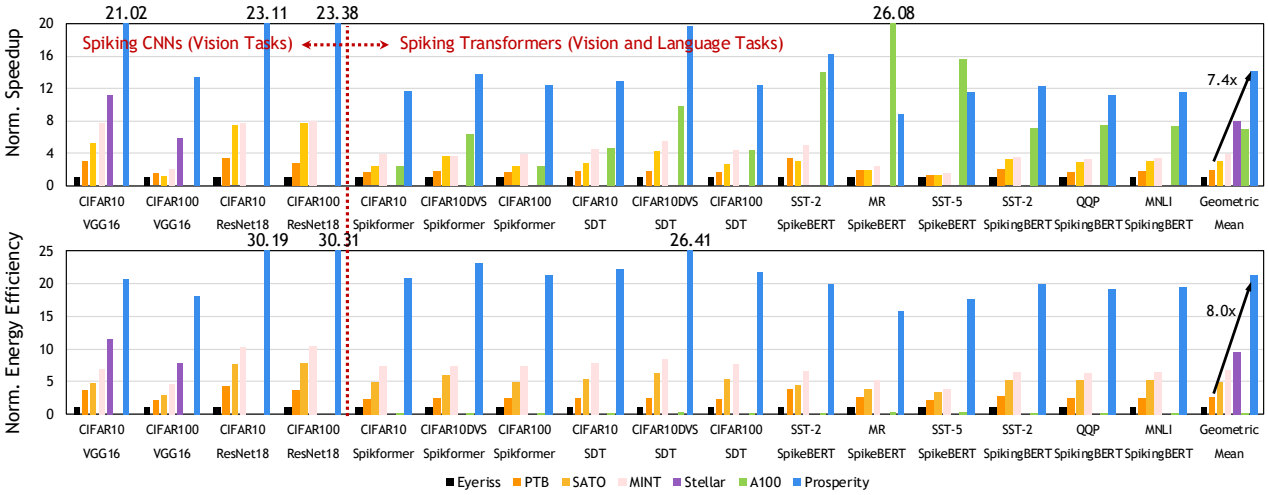

Fig. 8. Speedup and energy efficiency (normalized by Eyeriss) of *Prosperity*. *Prosperity* support both spiking CNNs and spiking transformers.

from workload imbalance problems. *Prosperity* has conquered these two problems through our dedicated Processor design. *Prosperity*'s row-wise dataflow with address decoder identifies the spikes and skips every zero, efficiently processing unstructured spike patterns in a single PE array. Significantly, *Prosperity* further reuses the prefix result to skip a significant amount of computation through ProSparsity. *Prosperity* achieves a $3.6\times$ speedup and $3.1\times$ energy efficiency over MINT. While MINT leverages BitSparsity and quantization (weights and membrane potentials), ProSparsity approach provides complementary benefits, enabling superior performance.

Compared with the algorithm-hardware co-design accelerator Stellar, *Prosperity* also achieves a $2.1\times$ speedup and $2.2\times$ energy efficiency on spiking CNNs even when Stellar has 31% more PEs. The speedup mainly comes from our significant improvement in sparsity. Stellar improved sparsity by modifying the SNN algorithm, while our proposed product sparsity improves the sparsity to a greater extent and in an algorithm-agnostic manner.

Comparison with A100. We compared with A100 on the end-to-end performance of various spiking transformers. Because of the distinctive and diverse operations of spiking transformers, spiking transformers are not efficiently supported by previous ASICs [52], [62] and neuromorphic chips [14]. Therefore, GPU is currently the better hardware for spiking transformers. *Prosperity* can achieve a $1.79\times$ speedup even though A100 has 312 TOPS peak throughput and 826 mm^2 area that is far more than *Prosperity* with only 0.529 mm^2. Notice that *Prosperity* has no significant speedup over A100 on SpikeBERT. This is because SpikeBERT is a relatively large model with 12 blocks of transformer encoder and 768 hidden dimension size. The minor speedup is caused by the better utilization of the computation core on A100. Nevertheless, *Prosperity* stands out in the competition for energy efficiency. *Prosperity* has $193\times$ energy efficiency improvement over A100. The reasons for this huge lead are twofold. A100's tensor core is designed for GeMM in DNN, which performs highly parallel MACs. However, spiking GeMM in SNN

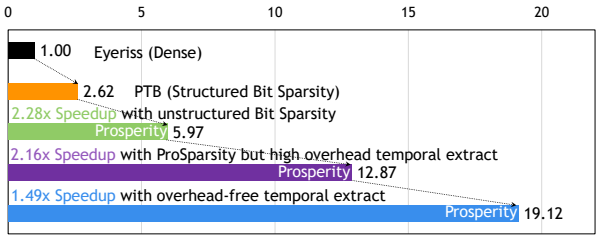

Fig. 9. Ablation study of *Prosperity*.

requires only accumulation, resulting in the underutilization of computation units in the tensor core. Moreover, *Prosperity* efficiently skips the zeros in spiking GeMM through bit sparsity and product sparsity, while GPU's SIMT architecture is rigid and unable to eliminate these redundancies.

D. Ablation Study: Speedup Analysis

We conduct an ablation study of each part of our design. This study is conducted on all evaluated models with average results. *Prosperity* efficiently handles the unstructured bit sparsity in spiking GeMM through our row-wise dataflow and an efficient address decoder. This unstructured sparsity processing achieves a $2.28\times$ speedup compared to PTB, which conducts structured sparsity processing.

We then introduce product sparsity in the accelerator but use the high-overhead Dispatcher method explained in Sec. V-D. This implementation utilizes product sparsity to reduce the number of accumulations but induces extra overhead to process rows in the correct order. It further achieves a $2.16\times$ speedup compared to simply using bit sparsity. However, this implementation does not fully unleash the potential of product sparsity.

Our proposed overhead-free dispatch elegantly removes the overhead of finding the processing order through our keen observation. It further achieves a $1.49\times$ speedup compared to the high overhead dispatching. Finally, compared to bit sparsity, *Prosperity* achieves an average of $3.2\times$ speedup for all models and up to $7.4\times$ speedup on SpikeBERT on SST-5.

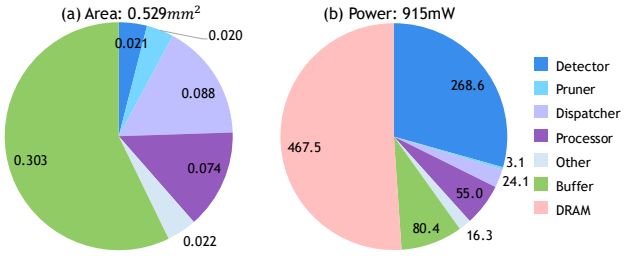

Fig. 10. *Prosperity* area and power breakdown.

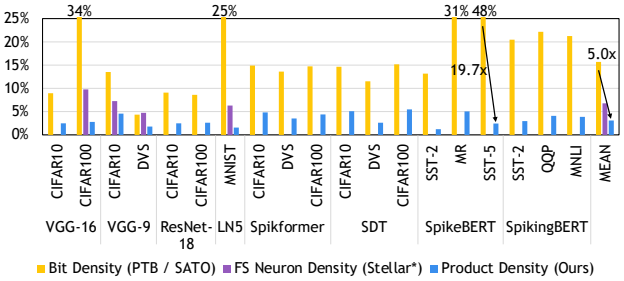

Fig. 11. Density comparison of different methods.

E. Area and Power Analysis

Area. The area and power overhead of different parts of *Prosperity* is shown in Fig. 10. For the area, we notice that the Dispatcher is the dominant part in *Prosperity*, except for the on-chip buffer. This is mainly attributed to the product sparsity table that maintains the product sparsity information for each row. The Processor with PE array also occupies the second largest part of the area.

Power. The power is evaluated on Spikformer with CIFAR10. DRAM power takes a large proportion of the total power. For on-chip power, we observe that the proportion of each part differs from the proportion of the area. The Detector with TCAM consumes most of the on-chip power because every cell in TCAM is activated to perform a highly parallel search every cycle, while the product sparsity table with a large area is only partially activated each cycle. The on-chip buffer and Processor also consume considerable energy. PPU (Detector, Pruner, Dispatcher, Processor) consumes most of the energy of *Prosperity*'s computation unit. This is reasonable since spiking GeMM is the bottleneck of SNNs, and we allocate most of the resources to spiking GeMM.

F. Sparsity Analysis

Activation-only Sparsity. Fig. 11 compares the density in bit sparsity, Stellar's FS neuron sparsity, and our product sparsity. Stellar introduced the use of FS neurons in SNNs to enhance sparsity, but this method sacrifices some accuracy of SNNs and has not been evaluated for applicability to emerging spiking transformers. In contrast, our product sparsity is algorithm-agnostic and can be applied to any SNN. Our product sparsity achieves up to a $19.7\times$ and an average $5.0\times$ reduction in density compared to bit sparsity. Therefore, ProSparsity on *Prosperity* brings a $3.2\times$ speedup compared to BitSparsity-only on *Prosperity*. There is a gap between the theoretic limit of $5.0\times$ speedup since EM ProSparsity has 100% sparsity but still takes one cycle to process. It also achieves an average $3.2\times$ density reduction compared to the algorithmic FS neuron method. Our product sparsity is effective for spike matrices with arbitrary density. Whether for relatively dense workloads (e.g., SpikeBERT) or relatively sparse workloads (ResNet18), we are able to reduce the density below 5%.

Dual-side Sparsity. We demonstrate the effectiveness of ProSparsity when applied to dual-side sparse SNN models proposed in LoAS. While LoAS focuses on BitSparsity and pruned sparse weights in SNNs, our ProSparsity introduces a novel, orthogonal direction for optimization. This complementary nature allows ProSparsity to be applied to LoAS-based accelerators, further reducing computation in dual-side sparse SNNs. To evaluate this synergy, we tested ProSparsity on three spiking CNNs pruned by LoAS. We measured three key metrics: weight density, original activation density, and activation density after applying ProSparsity. The results are presented in Tbl. V. LoAS effectively pruned the weights of these models to a density below 5% with minimum accuracy loss. By applying ProSparsity to these pruned SNNs, we achieved a significant reduction in activation density, reducing the activation density by $4.1\times$ on average.

G. Cost Trade-off Analysis in Prosperity

The overhead of SNN sparsity processing is generally necessary for all DNN accelerators. For instance, Stellar's sparsity processing overhead is 47% of its total energy and the computation is only 7% [62]. *Prosperity* architecture employs ProSparsity processing to reduce floating-point computations in spiking GeMM, introducing a trade-off between processing overhead and computational savings. Here, we provide a quantitative analysis, demonstrating that the benefits of ProSparsity outweigh its costs. The ProSparsity processing phase for a single tile involves:

- Bitwise operations in TCAM: $m^2 \times k$.
- Integer comparisons in the stable sorter: $2m \times \log(m)$.
- Integer comparisons in the pruner: $m + \log(m)$.

Consequently, TCAM bitwise operations have $O(m^2)$ complexity, dominating the ProSparsity processing overhead, so we can safely ignore the rest of the two terms. Using ProSparsity, we can reduce $\Delta S \times m \times k \times n$ floating-point additions, where ΔS denotes the increase in sparsity introduced by ProSparsity. Our experiments indicate that a floating-point addition incurs $45\times$ the hardware overhead of a single TCAM

TABLE V
DENSITY OF WEIGHT AND ACTIVATION IN LOAS [86] WITH PROSPARSITY.

Model	Tensor	LoAS	LoAS + *Prosperity*	Ratio
AlexNet	Weight	1.8%	1.8%	-
	Activation	29.32%	9.12%	3.21×
VGG-16	Weight	1.8%	1.8%	-
	Activation	31.07%	7.68%	4.05×
ResNet-19	Weight	4.0%	4.0%	-
	Activation	35.68%	6.96%	5.13×

bitwise operation. ProSparsity processing is more efficient than direct computation when the benefit-cost ratio exceeds one:

$$\frac{\Delta S \times m \times k \times n \times 45}{m^2 \times k \times 1} > 1,$$

where we can get the threshold as $\Delta S = 4.4\%$. That means that *Prosperity* can benefit when the sparsity increase is larger than 4.4%. Given our current tile size ($m = 256, k = 16, n = 128$) and average sparsity change in Sec. VII-F ($\Delta S = 13.35\%$), the benefit-cost ratio reach $3.0\times$. It confirms that our approach yields a favorable trade-off between processing overhead and computation reduction, thus validating the efficiency of our proposed method.

VIII. Discussion and Related Works

A. Architecture Scalability

Prosperity has the potential for good scalability. Both intra-PPU and inter-PPU parallelism could be leveraged to scale up the Prosperity architecture.

Intra-PPU. Intra-PPU parallelism could be achieved by simultaneously issuing multiple tasks (i.e., nodes in the ProSparsity tree) to the processor. There are no dependency for nodes in the same level. Therefore, we can scale to parallel process multiple nodes simultaneously.

Inter-PPU. Furthermore, Prosperity's scalability can be extended by exploring inter-PPU parallelism through deploying multiple PPUs within the system. Each PPU can process one tile of the spiking GeMM at a time, and large models can be divided into multiple tiles.

B. SNN Accelerators

Besides the work discussed in Sec. II-C, there have been extensive work utilizing the bit sparsity to efficiently accelerate SNNs [46], [58], [66], [86], [88]. SpinalFlow [66] is the first architecture designed for SNNs. It target on temporal coded SNNs to utilize BitSparsity and proposes time batching to reduce data movement. SATO [58] utilize bucket-sort to evenly distribute the task of sparse addition to multiple PEs. SATA [88] is a sparsity aware SNN accelerator that build upon systolic array, with a special support for SNN training.

Recent works have introduced DNN techniques like quantization and pruning to SNNs for more efficient accelerators. MINT [87], built upon SATA [88], incorporates weight and membrane potential quantization to reduce memory footprint. LoAS [86], the accelerator considering both spike bit sparsity and pruned weight sparsity, proposes a parallel dataflow for dual-side sparsity processing. Our ProSparsity introduces a novel sparsity paradigm in SNN activation, orthogonal to weight quantization and pruning, indicating potential for future integration with these techniques.

C. Efficient DNN

Quantitation and Sparsity. Compression techniques such as quantization [17], [20], [30], [32], [37], [56], [85] and sparsity [31], [34], [37], [51], [81], [93] are widely employed in DNN accelerators [9], [33], [35], [36], [79], [83], [89]–[91] to enhance both computational and memory efficiency for DNN models. Similar to LoAS [86] or MINT [87], these methods leverage pruning or quantization to compress weight tensors, thereby enabling further acceleration in SNN. Notably, *Prosperity* is fundamentally orthogonal to most quantization and pruning (sparsity) techniques, allowing it to remain compatible with quantized and sparse weights for continued computational optimization. This synergy opens up new avenues for future research.

Redundancy Removal. Redundancy removal methods have been applied in Graph Neural Networks [21], [47] (GNNs) accelerators [8], [24], categorized into offline and online approaches. Offline methods [44] perform global graph traversal to optimize redundancy elimination but incur heavy overhead, which is acceptable for static GNN graphs as it is a one-time process. In contrast, online methods [11], [24] use heuristic graph reordering to identify and eliminate redundancy locally, achieving efficiency in hardware but with less redundancy removed than offline methods.

However, these methods are GNN-specific and unsuitable for SNNs due to two main reasons. First, GNN methods leverage the presence of communities [28] in real-world graphs [5], [73], [92], which SNN sparsity patterns lack, exhibiting random distributions due to dynamic input activation. Second, GNNs typically handle highly sparse graphs ($\geq$99.99% sparsity) [1], [13], [24], whereas SNNs have lower sparsity (60%–90%) [55], [60], [94], making redundancy identification fundamentally different.

IX. Conclusion

In conclusion, this study introduces ProSparsity, a groundbreaking sparsity paradigm for SNNs that significantly enhances computational efficiency beyond traditional bit sparsity. By leveraging combinatorial similarities within spike matrices, ProSparsity enables substantial reductions in computations and energy consumption. Our proposed architecture, *Prosperity*, effectively addresses the spatial and temporal challenges of implementing ProSparsity, achieving linear time complexity and overhead-free sparse processing. The results demonstrate *Prosperity*'s superiority over state-of-the-art SNN accelerators, with average speedups of $7.4\times$ and $1.8\times$ compared to PTB and A100 GPU, respectively. Moreover, *Prosperity* achieves remarkable energy efficiency improvements of $8.0\times$ and $193\times$ over these benchmarks. These advancements not only push the boundaries of SNN acceleration but also open new avenues for efficient AI processing across diverse applications.

Acknowledgements

This work was supported in part by NSF grants 2328805 and 2112562, as well as Duke University's internal grant, the Beyond the Horizon Initiative. The authors thank the anonymous reviewers for their constructive feedback, which helped improve this work. We also extend our gratitude to Bowen Duan, Tergel Molom-Ochir, Jonathan Ku, Dr. Ziru Li, and Dr. Yangjie Zhou for their technical support and insightful discussions.

APPENDIX

A. Abstract

Our artifact is the simulator for *Prosperity*, which evaluates the runtime performance and energy.

We evaluate *Prosperity* on SNN models based on CNN model architecture or transformer architecture for image recognition and natural language processing tasks. For spiking CNN models, we have VGG16 and ResNet18. For spiking transformers, they include Spikformer, SDT, SpikeBERT, and SpikingBERT. These models are tested on eight dataset from NLP to image recognition, including SST-2, SST-5, MR, QQL, MNLI, CIFAR10, CIFAR100, CIFAR10-DVS. We provide the sparse activation matrix of models on these datasets, it would be used for architecture performance simulation. Our simulator evaluates the performance of *Prosperity* and baseline accelerators according to the provided sparse matrices. We run all the experiments on a Ubuntu server with an NVIDIA A100 GPU.

B. Artifact check-list (meta-information)

- **Compilation:** NVCC 12.1, GCC/G++ 11.3.0.
- **Model:** Spiking VGG, Spiking ResNet, Spikformer, Spike-Driven-Transformer, SpikeBERT, SpikingBERT.
- **Data set:** GLUE dataset, SST-5, MR, CIFAR10, CIFAR100, CIFAR10-DVS.
- **Run-time environment:** Ubuntu 22.04.2 LTS, CUDA 12.1, PyTorch 2.3.0.
- **Hardware:** A server with an x86_64 processor, an NVIDIA A100 GPU.
- **Output:** Architecture runtime performance, energy, matrix density.
- **How much disk space required (approximately)?:** 8GB.
- **How much time is needed to prepare workflow (approximately)?:** It takes about 30 minutes to prepare the environment.
- **How much time is needed to complete experiments (approximately)?:** It takes about 2 hours to run all of the experiments including architecture simulation, design space analysis, matrix sparsity analysis.
- **Publicly available?:** Our framework is publicly available on GitHub https://github.com/dubcyfor3/Prosperity
- **Code licenses (if publicly available)?:** MIT license.
- **Data licenses (if publicly available)?:** The datasets are publicly available through their original licensing terms.
- **Archived (provide DOI)?:** https://doi.org/10.5281/zenodo.14350604

C. Description

1) How to access: We archive the source code at https://doi.org/10.5281/zenodo.14350759. We recommend you access our GitHub repository: https://github.com/dubcyfor3/Prosperity for the latest version.

2) Hardware dependencies: A server equipped with an NVIDIA GPU.

3) Software dependencies: The experiments rely on the following software components.

- Ubuntu 22.04.2 LTS
- Python 3.10
- PyTorch 2.3.0
- Anaconda 24.1.2
- G++ 11.3.0
- CUDA 12.1
- CACTI 7.0

4) Data sets and models: The computer vision model including spiking VGG-16, ResNet-18, and Spikformer, Spike-Driven-Transformer models are run on image classification dataset CIFAR10, CIFAR100, CIFAR10-DVS. NLP models including SpikeBERT, SpikingBERT are run on NLP dataset GLUE (SST-2, MNLI, QQP), SST-5, MR.

D. Installation

We have well-documented README file to detail the installation instruction for each experiment at https://github.com/dubcyfor3/Prosperity.

E. Experiment workflow

The README file also specifies the detailed experiment workflow to get the results reported in paper.

F. Evaluation and expected results

Our artifact evaluate the runtime performance of our proposed Prosperity and baselines. It also include the design space exploration experiment, sparsity analysis, and buffer area evaluation.

G. Methodology

Submission, reviewing and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- https://cTuning.org/ae

REFERENCES

[1] L. Backstrom, D. Huttenlocher, J. Kleinberg, and X. Lan, "Group formation in large social networks: membership, growth, and evolution," in *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 44–54.

[2] M. Bal and A. Sengupta, "Spikingbert: Distilling bert to train spiking language models using implicit differentiation," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 10, 2024, pp. 10 998–11 006.

[3] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.

[4] K. E. Batcher, "Sorting networks and their applications," in *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, 1968, pp. 307–314.

[5] A. Broder, R. Kumar, F. Maghoul, P. Raghavan, S. Rajagopalan, R. Stata, A. Tomkins, and J. Wiener, "Graph structure in the web," *Computer networks*, vol. 33, no. 1-6, pp. 309–320, 2000.

[6] Y. Cao, Y. Chen, and D. Khosla, "Spiking deep convolutional neural networks for energy-efficient object recognition," *International Journal of Computer Vision*, vol. 113, pp. 54–66, 2015.

[7] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Tenth international workshop on frontiers in handwriting recognition*. Suvisoft, 2006.

[8] C. Chen, K. Li, Y. Li, and X. Zou, "Regnn: A redundancy-eliminated graph neural networks accelerator," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 429–443.

[9] F. Chen, L. Song, and Y. Chen, "Regan: A pipelined reram-based accelerator for generative adversarial networks," in *2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2018, pp. 178–183.

[10] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, "Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," *ACM SIGARCH Computer Architecture News*, vol. 42, no. 1, pp. 269–284, 2014.

[11] X. Chen, Y. Wang, X. Xie, X. Hu, A. Basak, L. Liang, M. Yan, L. Deng, Y. Ding, Z. Du *et al.*, "Rubik: A hierarchical architecture for efficient graph neural network training," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 4, pp. 936–949, 2021.

[12] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[13] G. Dai, Z. Zhu, T. Fu, C. Wei, B. Wang, X. Li, Y. Xie, H. Yang, and Y. Wang, "Dimmining: pruning-efficient and parallel graph mining on near-memory-computing," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 130–145.

[14] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *Ieee Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[15] L. Deng, G. Wang, G. Li, S. Li, L. Liang, M. Zhu, Y. Wu, Z. Yang, Z. Zou, J. Pei *et al.*, "Tianjic: A unified and scalable chip bridging spike-based and continuous neural computation," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 8, pp. 2228–2246, 2020.

[16] L. Deng, Y. Wu, X. Hu, L. Liang, Y. Ding, G. Li, G. Zhao, P. Li, and Y. Xie, "Rethinking the performance comparison between snns and anns," *Neural networks*, vol. 121, pp. 294–307, 2020.

[17] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm. int8 (): 8-bit matrix multiplication for transformers at scale," *arXiv preprint arXiv:2208.07339*, 2022.

[18] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen, L. Huang, H. Zhou, G. Li, and Y. Tian, "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, vol. 9, no. 40, p. eadi1480, 2023. [Online]. Available: https://www.science.org/doi/abs/10.1126/sciadv.adi1480

[19] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, "Deep residual learning in spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 21 056–21 069, 2021.

[20] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," 2023.

[21] T. Fu, C. Wei, Y. Wang, and R. Ying, "Desco: Towards generalizable and scalable deep subgraph counting," in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 2024, pp. 218–227.

[22] T. Fu, C. Wei, Z. Zhu, S. Yang, Z. Yu, G. Dai, H. Yang, and Y. Wang, "Clap: Locality aware and parallel triangle counting with content addressable memory," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–6.

[23] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, "Tangram: Optimized coarse-grained dataflow for scalable nn accelerators," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 807–820.

[24] T. Geng, C. Wu, Y. Zhang, C. Tan, C. Xie, H. You, M. Herbordt, Y. Lin, and A. Li, "I-gcn: A graph convolutional network accelerator with runtime locality enhancement through islandization," in *MICRO-54: 54th annual IEEE/ACM international symposium on microarchitecture*, 2021, pp. 1051–1063.

[25] W. Gerstner, "Time structure of the activity in neural network models," *Physical review E*, vol. 51, no. 1, p. 738, 1995.

[26] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press, 2014.

[27] S. Ghosh-Dastidar and H. Adeli, "Spiking neural networks," *International journal of neural systems*, vol. 19, no. 04, pp. 295–308, 2009.

[28] M. Girvan and M. E. Newman, "Community structure in social and biological networks," *Proceedings of the national academy of sciences*, vol. 99, no. 12, pp. 7821–7826, 2002.

[29] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[30] C. Guo, F. Cheng, Z. Du, J. Kiessling, J. Ku, S. Li, Z. Li, M. Ma, T. Molom-Ochir, B. Morris *et al.*, "A survey: Collaborative hardware and software design in the era of large language models," *arXiv preprint arXiv:2410.07265*, 2024.

[31] C. Guo, B. Y. Hsueh, J. Leng, Y. Qiu, Y. Guan, Z. Wang, X. Jia, X. Li, M. Guo, and Y. Zhu, "Accelerating sparse dnn models without hardware-support via tile-wise sparsity," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–15.

[32] C. Guo, Y. Qiu, J. Leng, X. Gao, C. Zhang, Y. Liu, F. Yang, Y. Zhu, and M. Guo, "SQuant: On-the-fly data-free quantization via diagonal hessian approximation," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=JXhROKNZzOc

[33] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, New York, NY, USA, 2023, pp. 1–15.

[34] C. Guo, F. Xue, J. Leng, Y. Qiu, Y. Guan, W. Cui, Q. Chen, and M. Guo, "Accelerating sparse dnns based on tiled gemm," *IEEE Transactions on Computers*, 2024.

[35] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.

[36] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: Efficient inference engine on compressed deep neural network," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 243–254, 2016.

[37] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.

[38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[39] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of physiology*, vol. 117, no. 4, p. 500, 1952.

[40] IEEE, "IEEE standard for SystemVerilog–unified hardware design, specification, and verification language," IEEE, New York, NY, USA, Standard 1800-2017, 2017.

[41] Intel Corporation, "Intel® sse4 programming reference," Intel Corporation, White Paper, 2007.

[42] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, 2023. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[43] E. M. Izhikevich, "Simple model of spiking neurons," *IEEE Transactions on neural networks*, vol. 14, no. 6, pp. 1569–1572, 2003.

[44] Z. Jia, S. Lin, R. Ying, J. You, J. Leskovec, and A. Aiken, "Redundancy-free computation for graph neural networks," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 997–1005.

[45] Z. Jonke, S. Habenschuss, and W. Maass, "Solving constraint satisfaction problems with networks of spiking neurons," *Frontiers in neuroscience*, vol. 10, p. 118, 2016.

[46] A. Khodamoradi, K. Denolf, and R. Kastner, "S2n2: A fpga accelerator for streaming spiking neural networks," in *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2021, pp. 194–205.

[47] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[48] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[49] H.-T. Kung, *Why systolic architecture?* Design Research Center, Carnegie-Mellon University, 1982.

[50] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[51] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage," in *Advances in Neural Information Processing Systems*, D. Touretzky, Ed., vol. 2. Morgan-Kaufmann, 1989.

[52] J.-J. Lee, W. Zhang, and P. Li, "Parallel time batching: Systolic-array acceleration of sparse spiking neural computation," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 317–330.

[53] H. Li, H. Liu, X. Ji, G. Li, and L. Shi, "Cifar10-dvs: an event-stream dataset for object classification," *Frontiers in neuroscience*, vol. 11, p. 309, 2017.

[54] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[55] Y. Li, Y. Guo, S. Zhang, S. Deng, Y. Hai, and S. Gu, "Differentiable spike: Rethinking gradient-descent for training spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 23 426–23 439, 2021.

[56] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," 2023.

[57] F. Liu, Z. Wang, W. Zhao, Y. Chen, T. Yang, X. Yang, and L. Jiang, "Randomize and match: Exploiting irregular sparsity for energy efficient processing in snns," in *2022 IEEE 40th International Conference on Computer Design (ICCD)*. IEEE, 2022, pp. 451–454.

[58] F. Liu, W. Zhao, Z. Wang, Y. Chen, T. Yang, Z. He, X. Yang, and L. Jiang, "Sato: spiking neural network acceleration via temporal-oriented dataflow and architecture," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 1105–1110.

[59] H. Liu, "Routing table compaction in ternary cam," *IEEE Micro*, vol. 22, no. 1, pp. 58–64, 2002.

[60] C. Lv, T. Li, J. Xu, C. Gu, Z. Ling, C. Zhang, X. Zheng, and X. Huang, "Spikebert: A language spikformer trained with two-stage knowledge distillation from bert," *arXiv preprint arXiv:2308.15122*, 2023.

[61] W. Maass, "Networks of spiking neurons: the third generation of neural network models," *Neural networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[62] R. Mao, L. Tang, X. Yuan, Y. Liu, and J. Zhou, "Stellar: Energy-efficient and low-latency snn algorithm and hardware co-design with spatiotemporal computation," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 172–185.

[63] S. K. Maurya and L. T. Clark, "A dynamic longest prefix matching content addressable memory for ip routing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 19, no. 6, pp. 963–972, 2010.

[64] W. I. D. Mining, *Introduction to data mining*. Springer, 2006.

[65] S. M. Mniszewski, "Graph partitioning as quadratic unconstrained binary optimization (qubo) on spiking neuromorphic hardware," in *Proceedings of the International Conference on Neuromorphic Systems*, 2019, pp. 1–5.

[66] S. Narayanan, K. Taht, R. Balasubramonian, E. Giacomin, and P.-E. Gaillardon, "Spinalflow: An architecture and dataflow tailored for spiking neural networks," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 349–362.

[67] NVIDIA, "Nvidia a100 tensor core gpu architecture," NVIDIA Corporation, Tech. Rep., 2020. [Online]. Available: https://www.nvidia.com/en-us/data-center/a100/

[68] K. Pagiamtzis and A. Sheikholeslami, "Content-addressable memory (cam) circuits and architectures: A tutorial and survey," *IEEE journal of solid-state circuits*, vol. 41, no. 3, pp. 712–727, 2006.

[69] B. Pang and L. Lee, "Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales," *arXiv preprint cs/0506075*, 2005.

[70] S. Park, S. Kim, B. Na, and S. Yoon, "T2fsnn: deep spiking neural networks with time-to-first-spike coding, in 2020 57th acm/ieee design automation conference (dac)," 2020.

[71] M. Pfeiffer and T. Pfeil, "Deep learning with spiking neurons: opportunities and challenges," *Frontiers in neuroscience*, vol. 12, p. 409662, 2018.

[72] F. Ponulak and A. Kasinski, "Introduction to spiking neural networks: Information processing, learning and applications," *Acta neurobiologiae experimentalis*, vol. 71, no. 4, pp. 409–433, 2011.

[73] S. Redner, "How popular is your paper? an empirical study of the citation distribution," *The European Physical Journal B-Condensed Matter and Complex Systems*, vol. 4, no. 2, pp. 131–134, 1998.

[74] A. Sengupta, Y. Ye, R. Wang, C. Liu, and K. Roy, "Going deeper in spiking neural networks: Vgg and residual architectures," *Frontiers in neuroscience*, vol. 13, p. 95, 2019.

[75] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[76] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proceedings of the 2013 conference on empirical methods in natural language processing*, 2013, pp. 1631–1642.

[77] C. Stöckl and W. Maass, "Optimized spiking neurons can classify images with high accuracy through temporal coding with two spikes," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 230–238, 2021.

[78] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "Glue: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.

[79] Y. Wang, C. Zhang, Z. Xie, C. Guo, Y. Liu, and J. Leng, "Dual-side sparse tensor core," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1083–1095.

[80] Z. Wang, Y. Fang, J. Cao, Q. Zhang, Z. Wang, and R. Xu, "Masked spiking transformer," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 1761–1771.

[81] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," *Advances in neural information processing systems*, vol. 29, 2016.

[82] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," *arXiv preprint arXiv:1704.05426*, 2017.

[83] B. Yan, Q. Yang, W.-H. Chen, K.-T. Chang, J.-W. Su, C.-H. Hsu, S.-H. Li, H.-Y. Lee, S.-S. Sheu, M.-S. Ho *et al.*, "Rram-based spiking nonvolatile computing-in-memory processing engine with precision-configurable in situ nonlinear activation," in *2019 Symposium on VLSI Technology*. IEEE, 2019, pp. T86–T87.

[84] M. Yao, J. Hu, Z. Zhou, L. Yuan, Y. Tian, B. Xu, and G. Li, "Spike-driven transformer," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[85] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "Zeroquant: Efficient and affordable post-training quantization for large-scale transformers," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 27 168–27 183. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/adf7fa39d65e2983d724ff7da57f00ac-Paper-Conference.pdf

[86] R. Yin, Y. Kim, D. Wu, and P. Panda, "Loas: Fully temporal-parallel datatflow for dual-sparse spiking neural networks," *arXiv preprint arXiv:2407.14073*, 2024.

[87] R. Yin, Y. Li, A. Moitra, and P. Panda, "Mint: Multiplier-less integer quantization for energy efficient spiking neural networks," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 830–835.

[88] R. Yin, A. Moitra, A. Bhattacharjee, Y. Kim, and P. Panda, "Sata: Sparsity-aware training accelerator for spiking neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 6, pp. 1926–1938, 2022.

[89] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.

[90] A. H. Zadeh, M. Mahmoud, A. Abdelhadi, and A. Moshovos, "Mokey: enabling narrow fixed-point inference for out-of-the-box floating-point transformer models," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, New York, NY, USA, 2022, pp. 888–901.

[91] C. Zhang, Y. Wang, Z. Xie, C. Guo, Y. Liu, J. Leng, G. Sun, Z. Ji, R. Wang, Y. Xie *et al.*, "Dstc: Dual-side sparsity tensor core for dnns acceleration on modern gpu architectures," *IEEE Transactions on Computers*, 2024.

[92] J. Zhang, H. Wang, Q. Ding, J. Gu, R. Assouly, W. D. Oliver, S. Han, K. R. Brown, H. Li, Y. Chen *et al.*, "Qplacer: Frequency-aware component placement for superconducting quantum computers," *arXiv preprint arXiv:2401.17450*, 2024.

[93] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H$_2$o: Heavy-hitter oracle for efficient generative inference of large language models." [Online]. Available: http://arxiv.org/abs/2306.14048

[94] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. Yan, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," *arXiv preprint arXiv:2209.15425*, 2022.